

Sistemas De Computação

João Eduardo Montandon
DCC024

```
#include <stdio.h>

int i;
int main() {
    for (i = 1; i <= 100; i++)
        printf("%d ", i);
}
```

```
$ gcc hello.c
$ ./a.out
1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28
29 30 31 32 33 34 35 36 37 38 39 40 41 42 ...
```

O que aconteceu, **exatamente**?

"Compilando" Um Programa

Por debaixo dos panos, um processo muito mais detalhado ocorreu quando "compilamos" e "executamos" um programa

- Como a máquina reconhece `i`?
 - De onde veio o `printf()`?
 - Em qual local da memória o programa foi carregado?
-

1. Criação Do Arquivo

Desenvolvedor cria o arquivo de código-fonte com o programa a ser desenvolvido

```
#include <stdio.h>

int i;
int main() {
    for (i = 1; i <= 100; i++)
        printf("%d ", i);
}
```

- Linguagem adotada é de alto nível, geralmente
- Alto nível: 👍 Humanos / 👎 Máquina

2. Compilação (Compiling)

🔗 Traduzir programas em uma linguagem de mais baixo nível (Assembly)

```
$ gcc hello.c -S
```

```
...
i:
    .zero    4
    .section      .rodata
main:
    movl    i(%rip), %eax
    movl    %eax, %esi
    leaq    .LC0(%rip), %rax
    movq    %rax, %rdi
    movl    $0, %eax
    call    printf@PLT
    movl    i(%rip), %eax
    addl    $1, %eax
    movl    %eax, i(%rip)
```

-
- Específico para a máquina onde código foi compilado
 - Cada linha representa um dado ou uma instrução de máquina
 - Programas eram criados diretamente em Assembly até 1957!

3. Assembling

```
$ as hello.s -o main.o
```



Por que esta etapa?

- Assembly ainda é texto!
 - Ainda usa-se nomes, ao invés de endereços de memória
- Linguagem de máquina deve estar em formato binário!

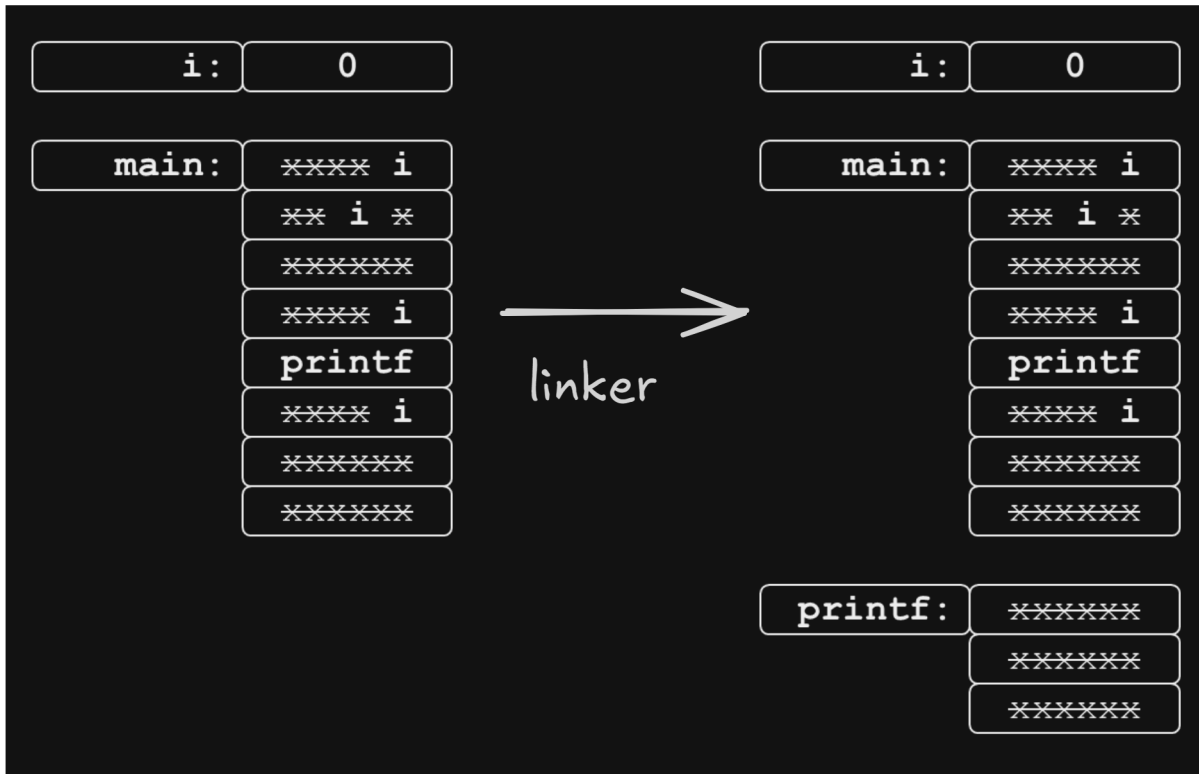
4. Ligação (Linking)

- Arquivo objeto ainda não é executável
 - Faltam algumas partes (`printf`)
- Essa etapa procura por partes do programa que foram definidos "externamente",
 - Bibliotecas do sistema
 - Funções do desenvolvedor

- Diferentes estratégias de junção podem ser aplicadas
 - Substituição X Referenciação

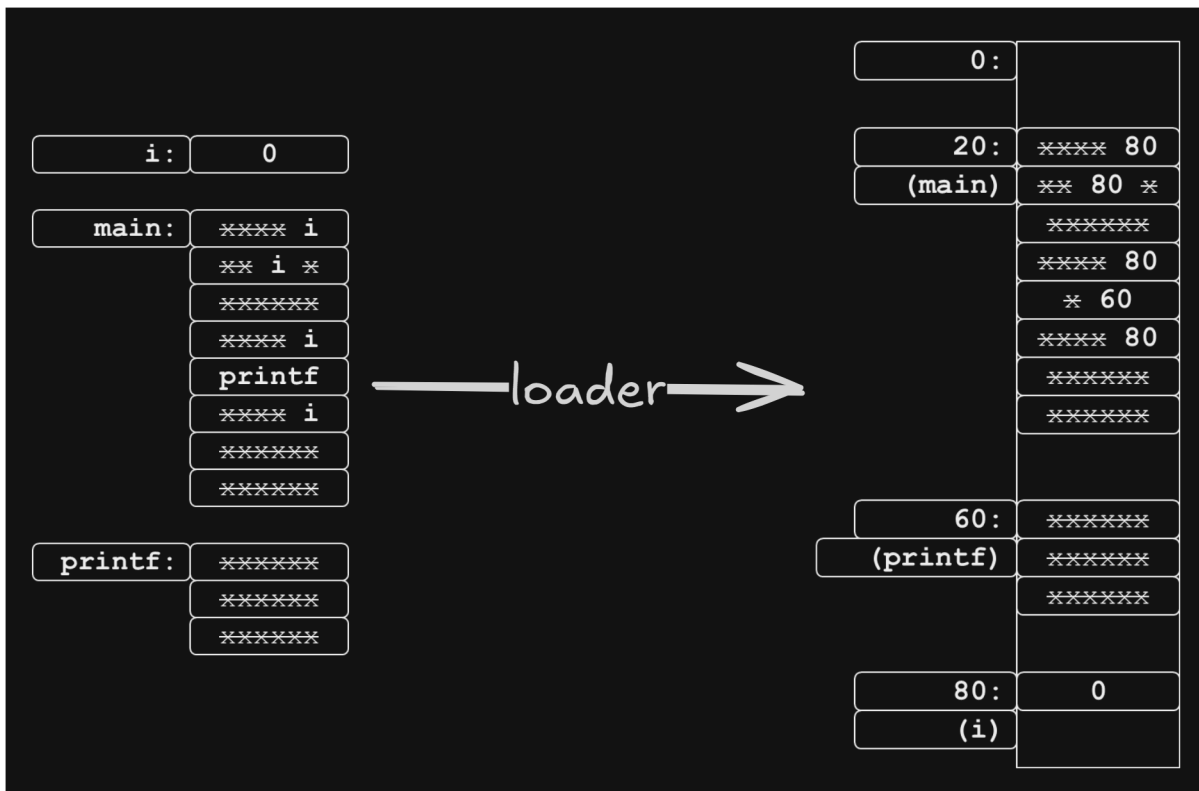
🎯 Incluir código externo ao programa em processamento

```
$ ld main.o -o main.out
```



5. Carregamento (Loading)

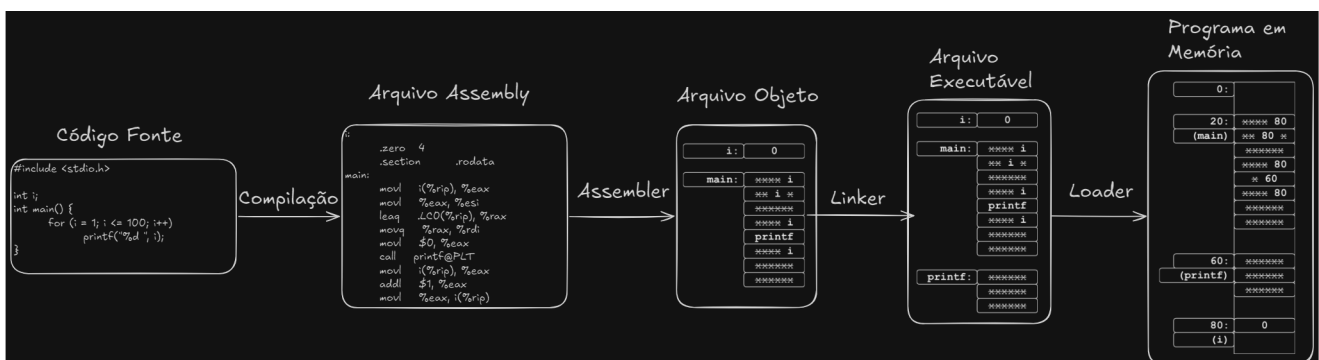
- Executável ainda não pode ser executado... 🤖
 - Ainda possui alguns nomes
- Quando o programa é executado, o loader irá posicionar na memória todos os "nomes" ainda presentes no programa



6. Execução

- Todo programa agora é linguagem de máquina
- Processador executa instrução por instrução

1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28
29 30 31 32 33 34 35 36 37 38 39 40 41 42 ...



Otimização

- Os compiladores atualmente apresentam diversas rotinas de melhoria

- Objetivo é tornar o código mais eficiente, menor, ou ambos!
 - Pode ser feita em diversas etapas (assembler, linker, loader, ...)
 - Isso pode fazer com que programas escritos de forma distinta gerem a mesma linguagem de máquina!
-

```
int i = 0;
int x = 20;
while (i < 100) {
    a[i++] = x*x*x;
}
```

```
int i = 0;
int temp = x*x*x;
while (i < 100) {
    a[i++] = temp;
}
```

Qual deles é mais eficiente?

- Remoção de *loop invariant*
 - Outras possíveis melhorias:
 - duplicação de código (*specialized copy*)
 - Remoção de *dead code*
-

Variações

- Naturalmente, esse processo não é seguido a risca por todas as linguagens
 - Mudanças introduzidas fazem parte da proposta da linguagem
-

Interpretadores

```
x = 14
y = 49
while x != y:
    if x < y:
        y = y - x
```

```
else:
    x = x - y;
print(x)
```

Qual o valor de `x` ?

- Você compilou para assembly?
- Gerou linguagem de máquina?
- Criou endereços para as variáveis?

Ou simplesmente executou o programa?

- Interpretar um programa consiste em executar suas instruções, sem trabalhar com alguma linguagem intermediária
 - Interpretadores costumam ser mais lentos
 - Compilação: Apenas a execução é feita em *runtime*
 - Interpretação: Todo processo é feito em *runtime*
-

Isso tem alguma vantagem? 🤔

Máquinas Virtuais

- "Meio-termo" entre compilação e interpretação
 - Compila-se o código não para a linguagem de máquina , mas sim para uma linguagem intermediária
 - A linguagem intermediária será interpretada por uma máquina virtual
-

Para quê? 🤔

1. Estamos interpretando uma linguagem de baixo nível (⬆ desempenho)
 2. Nosso código deve compliant apenas com a Máquina Virtual (⬆ portabilidade)
-

Exemplo: Java

```
public class Hello {  
    public static void main(String[] args) {  
        for(int i = 1; i <= 100; i++)  
            System.out.print(i + " ");  
    }  
}
```

```
$ javac Hello.java  
$ javap -c Hello
```

Variações

- linguagem intermediária = código implementado ( puramente interpretado)
- linguagem intermediária = fluxo de tokens
- linguagem intermediária = linguagem da máquina virtual
- linguagem intermediária = linguagem de máquina ( puramente compilado)

Delayed Linking

Consiste em atrasar o processo de ligação para que funções de terceiros não sejam incluídas no executável do programa

- Windows: Funções armazenadas em arquivos `.dll` (dynamic-link library)
- Unix: Funções armazenadas em arquivos `.so` (shared object)

-
1. Durante o carregamento: Busca pelo arquivo e liga o programa logo antes da execução
 2. Durante a execução: O programa faz chamadas explícitas ao SO para buscar o arquivo e carregar as funções necessárias
-

Por que usar?

- Múltiplos programas podem compartilhar exatamente o mesmo arquivo de biblioteca

- As bibliotecas podem ser atualizadas sem necessidade de atualizar o programa (🤖)
 - As bibliotecas podem ser carregadas somente quando utilizadas
-

Compilação Dinâmica

🔗 Incluir partes do processo de compilação após execução do programa

- Compilar uma função apenas quando chamada
 - Compilar apenas trechos chamados recorrentemente
 - Compilar tudo para linguagem intermediária, e para máquina apenas as partes frequentes (JIT)
-

Binding

```
int i;
int main() {
    for (i = 1; i <= 100; i++)
        printf("%d ", i);
}
```

- Quais valores `int` suporta?
 - Qual o tipo de `printf`?
 - Qual o endereço de `main`?
 - Qual o valor de `i`?
-

🔗 Processo que consiste em associar as propriedades aos identificadores da linguagem

Feito em etapas distintas

1. Tempo de Definição
 2. Tempo de Implementação
 3. Tempo de Compilação
 4. Tempo de Ligação
 5. Tempo de Carregamento
 6. Tempo de Execução
-

1. Tempo De Definição

Significado das palavras reservadas da linguagem: `for`, `void`, etc

2. Tempo De Implementação

Algumas propriedades associadas às palavras reservadas

- Faixa de valores associadas ao tipo `int` em C
 - Em Java, definidos na etapa anterior
 - Atribuição de valores máximos/mínimos
 - Tamanho do identificador, dimensão de arrays, etc
-

3. Tempo De Compilação

Propriedades relacionadas a estrutura do programa implementado

- Tipos das variáveis
- Declaração das variáveis

4. Tempo De Ligação

Propriedades relacionadas a elementos externos do programa compilado

- Referências a funções e objetos externos
-

5. Tempo De Carregamento

Propriedades relacionadas a localização dos elementos do programa

- Endereço de memória para funções
- Endereço de memória para variáveis estáticas

6. Tempo De Execução

Definidas quando o programa entra em execução

- Valores de variáveis
- Tipos de variáveis, em linguagens de tipagem dinâmicas

Late Binding VS Early Binding

- Late binding: Traz mais flexibilidade a linguagem durante a execução do programa (tipagem dinâmica, escopo dinâmico, etc)
 - Early binding: Mais rápida e mais segura durante a execução do programa (tipagem estática, escopo de bloco, etc)
-

Outras Características

- Debuggers: linguagens oferecem recursos estáticos (*memory dump*) e dinâmicos (*step-by-step*)
 - Suporte *runtime*: recursos adicionais incluídos pelo *linker*, mesmo que não exigido pelo programa
 - Processo de inicialização
 - Tratamento de exceção
 - Gerenciamento de memória
 - Interface com SO (I/O, concorrência, etc)
-

Conclusões

- Sistemas de Computação: ecossistema responsável por implementar e executar programas
- Hoje: Uma rápida introdução
- Tópicos serão abordados mais em profundidade no decorrer do curso